Assignment -11.1

Name: Akshaya Nemalipuri
Batch: 37
Roll No: 2303A2441
Lab: 11.1

Task Description -1 – (Stack Using AI Guidance)

• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.
Expected Output:
• A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

Prompt:

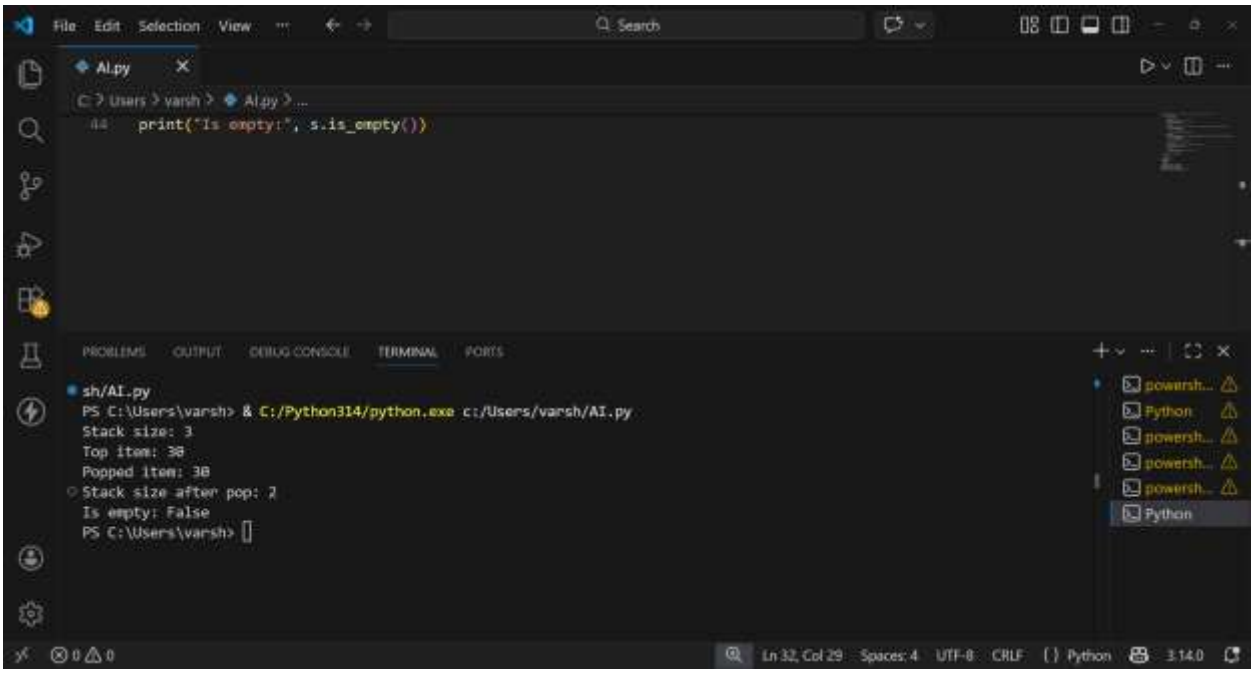
Design and implement a Stack data structure supporting basic stack operations.

Code:

```
# design and implement a Stack data structure supporting basic stack operations.
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an
error if the stack is empty."""
        if not self.is_empty():
            return self.stack.pop()
```

```
        else:
            raise IndexError("Stack is empty")

    def peek(self):
        """Return the item at the top of the stack without removing it.
Raises an error if the stack is empty."""
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Stack is empty")

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.stack) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.stack)

# Create a stack and test it
s = Stack()

# Push some items
s.push(10)
s.push(20)
s.push(30)

print("Stack size:", s.size())
print("Top item:", s.peek())
print("Popped item:", s.pop())
print("Stack size after pop:", s.size())
print("Is empty:", s.is_empty())
```

Output:



Explanation:

This code implements a **Stack** — a Last-In-First-Out (LIFO) data structure where items are added and removed from the same end (top).

Methods:

__init__() - Initializes an empty stack using a list
push(item) - Adds an item to the top of the stack
pop() - Removes and returns the top item; raises an error if empty
peek() - Returns the top item without removing it
is_empty() - Checks if the stack has no items
size() - Returns the number of items in the stack

Task Description -2 – (Queue Design)

• Task:Use AI assistance to create a Queue data structure following FIFO principles

Expected Output:

• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

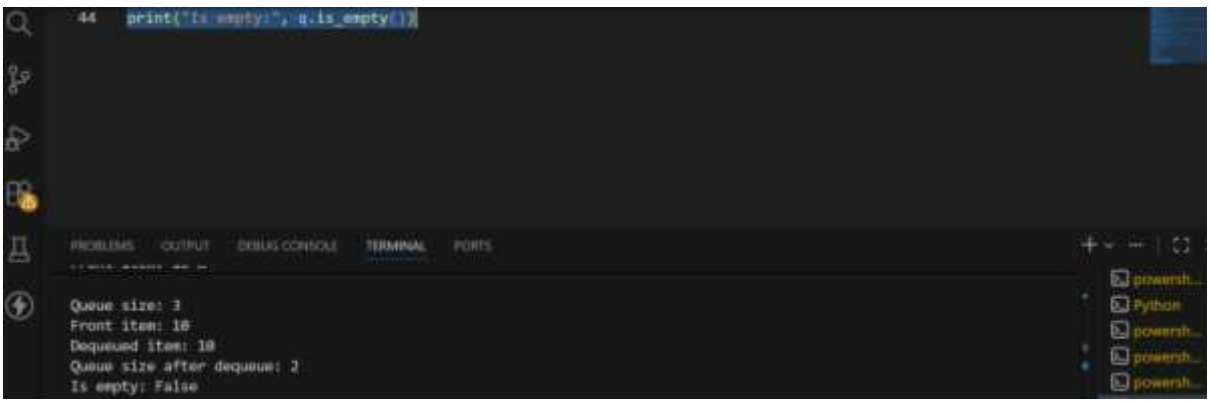
Prompt: **create a Queue data structure following FIFO principles**

Code:

```
#create a Queue data structure following FIFO principles
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.queue.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an
error if the queue is empty."""
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("Queue is empty")

    def front(self):
        """Return the item at the front of the queue without removing it.
Raises an error if the queue is empty."""
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("Queue is empty")

    def is_empty(self):
        """Return True if the queue is empty, False otherwise."""
        return len(self.queue) == 0

    def size(self):
        """Return the number of items in the queue."""
        return len(self.queue)

# Create a queue and test it
q = Queue()
```

```python
# Enqueue some items
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

print("Queue size:", q.size())
print("Front item:", q.front())
print("Dequeued item:", q.dequeue())
print("Queue size after dequeue:", q.size())
print("Is empty:", q.is_empty())
```

Output:



Explanation of the code:

This code implements a Queue — a First-In-First-Out (FIFO) data structure where items are added at the rear and removed from the front.

Methods:

- __init__() - Initializes an empty queue using a list enqueue(item) - Adds an item to the rear of the queue
- dequeue() - Removes and returns the item at the front; raises an error if empty
- front() - Returns the front item without removing it
- is_empty() - Checks if the queue has no items
- size() - Returns the number of items in the queue

Task Description -3 –(Singly Linked List Construction)

• Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

• Correctly functioning linked list with node creation, insertion logic, and display functionality.

Code:

```
#build a singly linked list supporting insertion and traversal
class Node:
     def __init__(self, data):
        """Initialize a node with data and next pointer."""
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert_at_beginning(self, data):
        """Insert a node at the beginning of the list."""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        """Insert a node at the end of the list."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
```

```python
    def insert_at_position(self, data, position):
        """Insert a node at a specific position (0-indexed)."""
        if position == 0:
            self.insert_at_beginning(data)
            return

        new_node = Node(data)
        current = self.head

        for _ in range(position - 1):
            if not current:
                raise IndexError("Position out of range")
            current = current.next

        if not current:
            raise IndexError("Position out of range")

        new_node.next = current.next
        current.next = new_node

    def traverse(self):
        """Traverse and print all elements in the list."""
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        return " -> ".join(elements) if elements else "Empty list"

    def display(self):
        """Display the linked list."""
        print(self.traverse())


# Test the singly linked list
print("=== Singly Linked List Demo ===\n")

ll = SinglyLinkedList()
```

```python
# Insert at beginning
ll.insert_at_beginning(10)
ll.insert_at_beginning(5)
print("After inserting 10, then 5 at beginning:")
ll.display()

# Insert at end
ll.insert_at_end(20)
ll.insert_at_end(30)
print("\nAfter inserting 20, then 30 at end:")
ll.display()

# Insert at specific position
ll.insert_at_position(15, 2)
print("\nAfter inserting 15 at position 2:")
ll.display()

# Traverse the list
print("\nFinal list traversal:")
ll.display()
```

Output:

Explanation:

Classes: Node - Represents each element in the list data - stores the value next - points to the next node (or None if it's the last node) SinglyLinkedList - Manages the list head - points to the first node Methods: insert_at_beginning(data) - Adds a node at the front by making it the new head insert_at_end(data) - Traverses to the last node and appends a new node insert_at_position(data, position) - Inserts at a specific index (0-based) traverse() - Walks through all nodes and returns them as a formatted string display() - Prints the list

Task Description -4 – (Binary Search Tree Operations)

• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

• BST program with correct node insertion and in-order traversal output.

Prompt: Implement a Binary Search Tree with AI support focusing on insertion and traversal

Code:

```
# Implement a Binary Search Tree with AI support focusing on insertion and traversal
class TreeNode:
    def __init__(self, data):
        """Initialize a tree node with data and left/right pointers."""
        self.data = data
        self.left = None
        self.right = None


class BinarySearchTree:
    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, data):
```

```
        """Insert a value into the BST maintaining the BST property."""
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Helper method to recursively insert data into the tree."""
        if data < node.data:
            # Insert into left subtree
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            # Insert into right subtree
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)

    def inorder_traversal(self):
        """Traverse the tree in inorder (Left, Root, Right) and return as
list."""
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        """Helper method for inorder traversal."""
        if node:
            self._inorder_recursive(node.left, result)
            result.append(node.data)
            self._inorder_recursive(node.right, result)

    def preorder_traversal(self):
        """Traverse the tree in preorder (Root, Left, Right) and return as
list."""
        result = []
        self._preorder_recursive(self.root, result)
```

```
        return result

    def _preorder_recursive(self, node, result):
        """Helper method for preorder traversal."""
        if node:
            result.append(node.data)
            self._preorder_recursive(node.left, result)
            self._preorder_recursive(node.right, result)

    def postorder_traversal(self):
        """Traverse the tree in postorder (Left, Right, Root) and return
as list."""
        result = []
        self._postorder_recursive(self.root, result)
        return result

    def _postorder_recursive(self, node, result):
        """Helper method for postorder traversal."""
        if node:
            self._postorder_recursive(node.left, result)
            self._postorder_recursive(node.right, result)
            result.append(node.data)

    def search(self, data):
        """Search for a value in the BST."""
        return self._search_recursive(self.root, data)

    def _search_recursive(self, node, data):
        """Helper method to recursively search for data."""
        if node is None:
            return False
        if data == node.data:
            return True
        elif data < node.data:
            return self._search_recursive(node.left, data)
        else:
            return self._search_recursive(node.right, data)

    def find_min(self):
        """Find the minimum value in the BST."""
```

```python
        if self.root is None:
            return None
        current = self.root
        while current.left:
            current = current.left
        return current.data

    def find_max(self):
        """Find the maximum value in the BST."""
        if self.root is None:
            return None
        current = self.root
        while current.right:
            current = current.right
        return current.data


# Test the Binary Search Tree
print("=== Binary Search Tree Demo with AI Support ===\n")

bst = BinarySearchTree()

# Insert values
values = [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]
print(f"Inserting values: {values}")
for val in values:
    bst.insert(val)

print("\nTraversals:")
print(f"Inorder (sorted):   {bst.inorder_traversal()}")
print(f"Preorder:           {bst.preorder_traversal()}")
print(f"Postorder:          {bst.postorder_traversal()}")

print("\nSearching & Min/Max:")
print(f"Search for 35: {bst.search(35)}")
print(f"Search for 100: {bst.search(100)}")
print(f"Minimum value: {bst.find_min()}")
print(f"Maximum value: {bst.find_max()}")

print("\n=== BST Structure ===")
```

```
print("        50")
print("       /  \\")
print("     30   70")
print("    / \\  / \\")
print("  20 40 60 80")
print(" / \\  \\ \\")
print("10 25 35 65")
```

Output:

```
=== BST Structure ==, 40, 30, 65, 60, 80, 70, 50]

Searching & Min/Max:
Search for 35: True
=== BST Structure ===
        50
       /  \
     30   70
    / \  / \
   20 40 60 80
  / \  \ \
 10 25 35 65
```

Explanation:

TreeNode - Represents individual nodes data - stores the value left - points to smaller values right - points to larger values BinarySearchTree - Manages the BST root - points to the top node Key Methods: insert(data) - Adds values maintaining BST property (left < parent < right) Uses recursion to find correct position Smaller values go left, larger go right inorder_traversal() - Left → Root → Right Returns values in sorted order (10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80) preorder_traversal() - Root → Left → Right Visits root first, useful for copying trees postorder_traversal() - Left → Right → Root Visits root last, useful for deleting trees search(data) - Finds if a value exists using binary

Task Description -5 – (Hash Table Implementation)

• Task: Create a hash table using AI with collision handling

Expected Output:

• Hash table supporting insert, search, and delete using chaining

or open

Prompt: Create a hash table with collision handling

Code:

```
#Create a hash table with collision handling

class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value)
                    return
            self.table[index].append((key, value))

    def search(self, key):
        index = self.hash_function(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        return None

    def delete(self, key):
        index = self.hash_function(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i]
```

```
                    return True
        return False
# Example usage
hash_table = HashTable(10)
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
print(hash_table.search("name"))  # Output: None
```

Output:

```
Alice
30
None
```

Explanation of code:

Purpose:Implements a hash table using separate chaining (each bucket is a list of (key, value) pairs) to handle collisions.

- __init__(self, size): Initializes self.size (number of buckets) and self.table (list of buckets, initially None entries). Buckets are created lazily (a bucket becomes a list when first used).
- hash_function(self, key): Computes hash(key) % self.size to map any hashable key to a bucket index.
- insert(self, key, value): Computes bucket index. If bucket is None, creates a list and appends (key, value). If bucket exists, scans for an existing key: if found, updates value; otherwise appends the new pair. Collision handling: multiple keys that hash to the same index live in the same bucket list.
- Search(self, key): Computes index and scans the bucket list (if present). Returns the associated value if found, else returns None.
- delete(self, key): Computes index and scans the bucket list. Removes the matching pair if found and returns True; otherwise returns False.